

EE45: Intro to Foundations of ML

Mod 1 - Lec 4: Data Fitting Concepts Part IV

Overview: [Module 1] Data Fitting Concepts

- Last Week: Vector notation, norms products, loss functions from norms
- Today:
 - ▶ review k -means clustering, nearest neighbors, document dissimilarity
 - ▶ standardization of data

References:

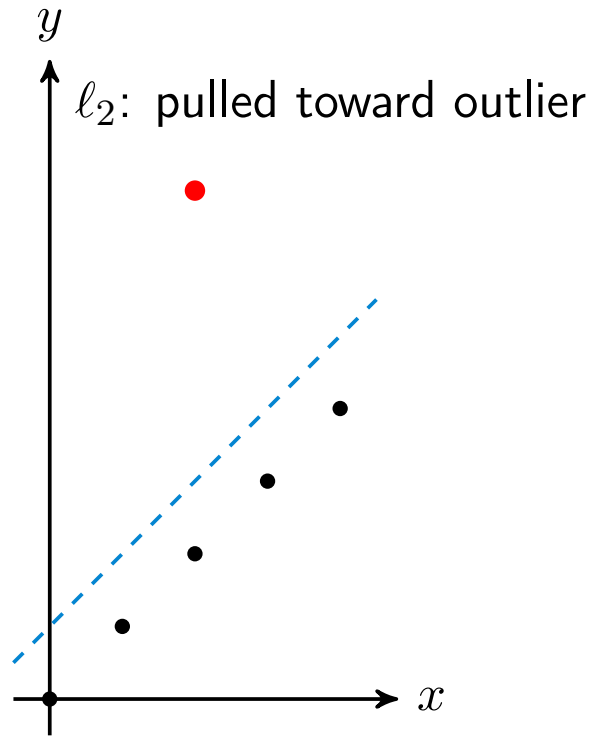
- [VMLS]: Chapters 1–3
- Topics: Vectors, Inner product, Linear functions, Regression model, Norm and distance

Content Summary

ie. norms $\| \cdot \|$

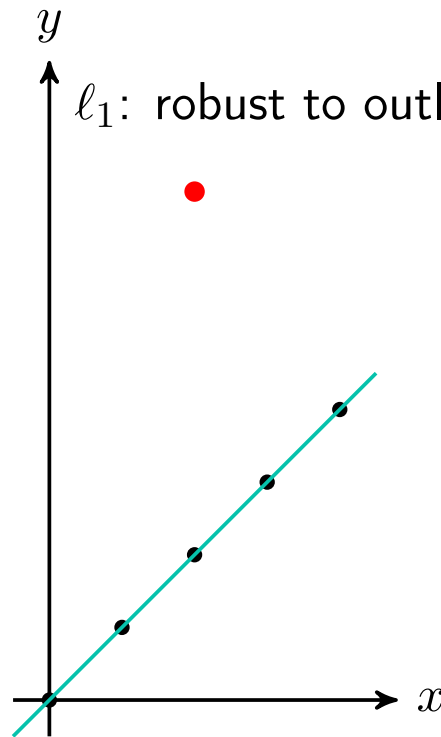
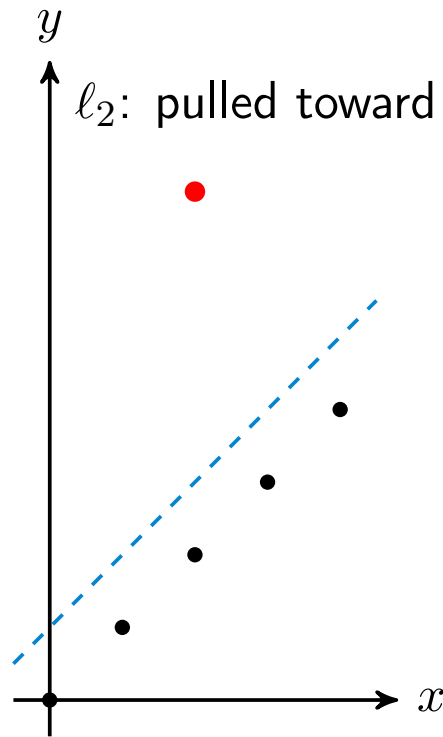
- Main Concepts: ML predictors using distance measures only
 - ▶ k-means
 - ▶ k-Nearest Neighbors (k-NN)
 - ▶ STANDARDIZATION OF DATA

Fitting a line with one extreme outlier: ℓ_2 vs ℓ_1 vs ℓ_∞



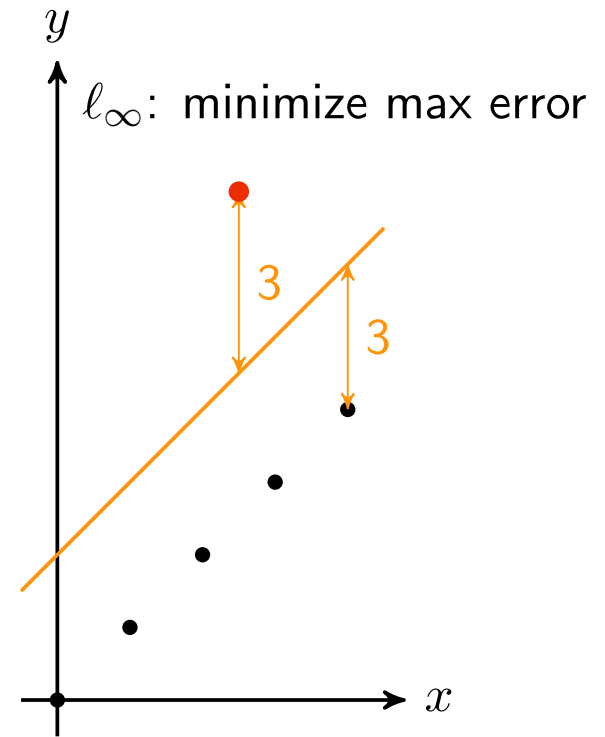
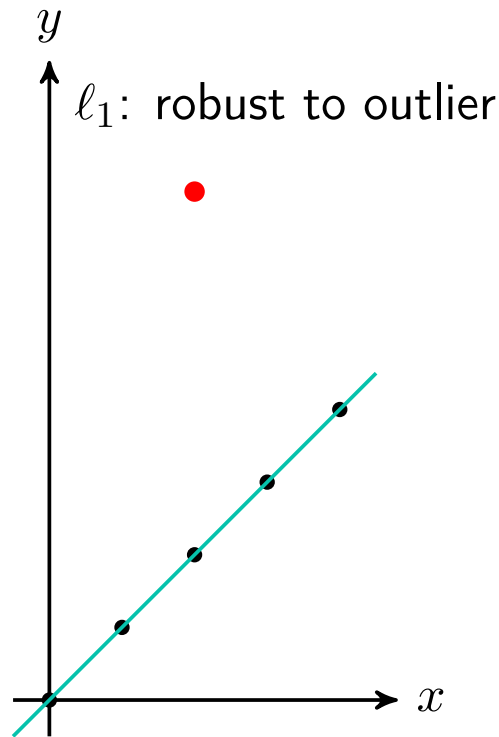
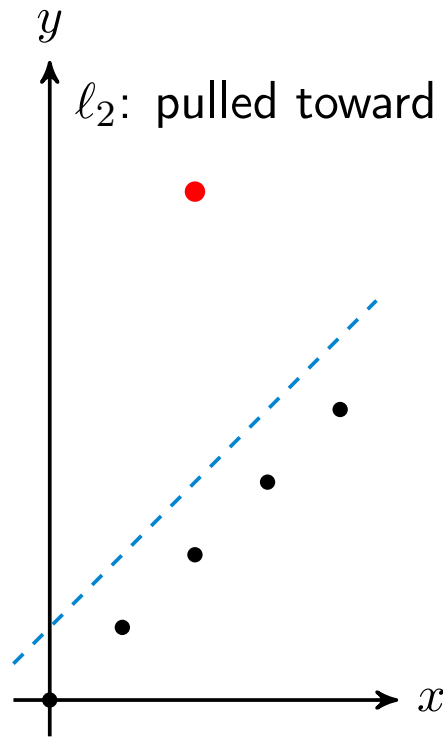
- ℓ_2 : Large errors dominate (quadratic), so the fit bends toward the outlier.
- ℓ_1 : Linear growth; outlier has limited pull, line stays with the bulk.
- ℓ_∞ : *minimizes the largest absolute residual*

Fitting a line with one extreme outlier: ℓ_2 vs ℓ_1 vs ℓ_∞



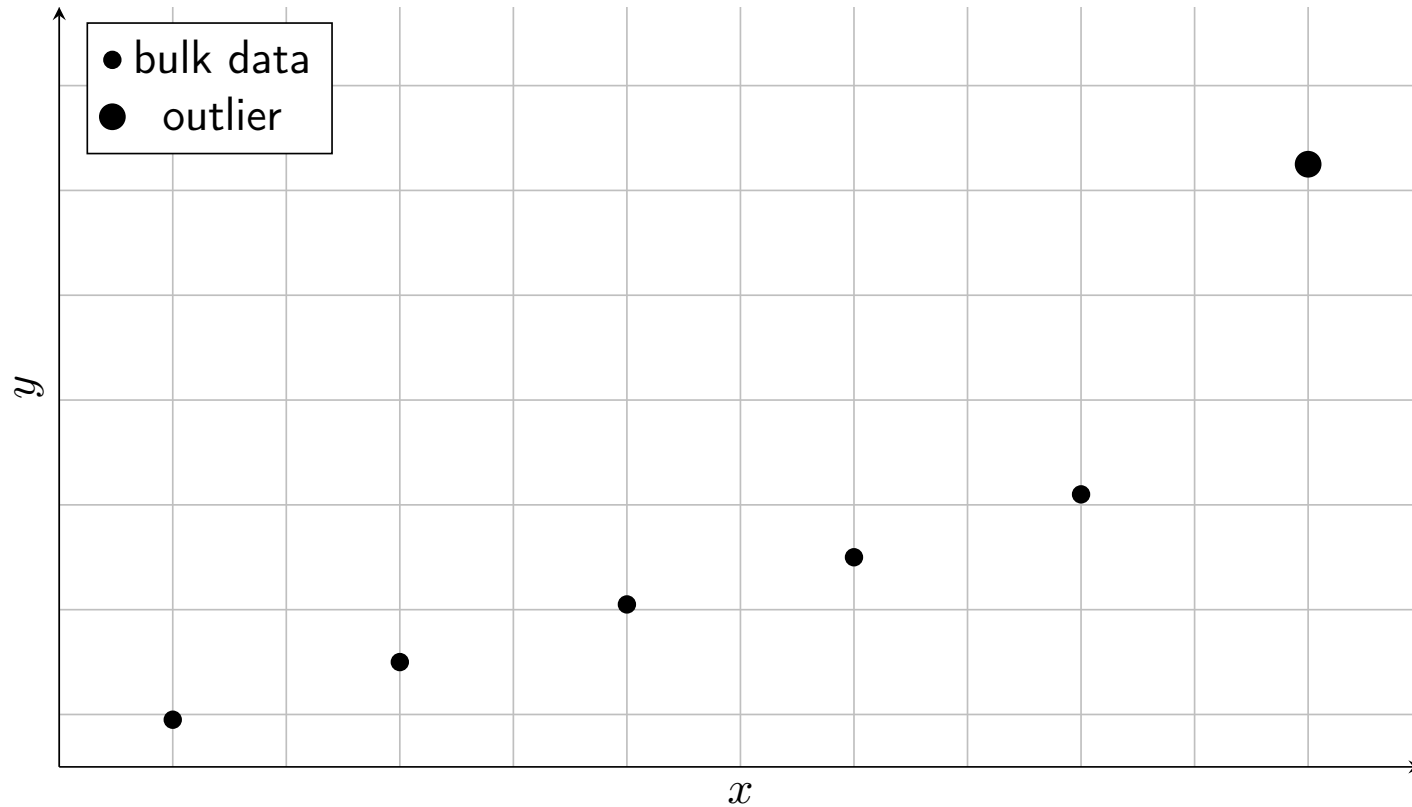
- ℓ_2 : Large errors dominate (quadratic), so the fit bends toward the outlier.
- ℓ_1 : Linear growth; outlier has limited pull, line stays with the bulk.
- ℓ_∞ : *minimizes the largest absolute residual*

Fitting a line with one extreme outlier: ℓ_2 vs ℓ_1 vs ℓ_∞



- ℓ_2 : Large errors dominate (quadratic), so the fit bends toward the outlier.
- ℓ_1 : Linear growth; outlier has limited pull, line stays with the bulk.
- ℓ_∞ : *minimizes the largest absolute residual*

Choosing Norm Impacts Model Fit: Let's apply the logic



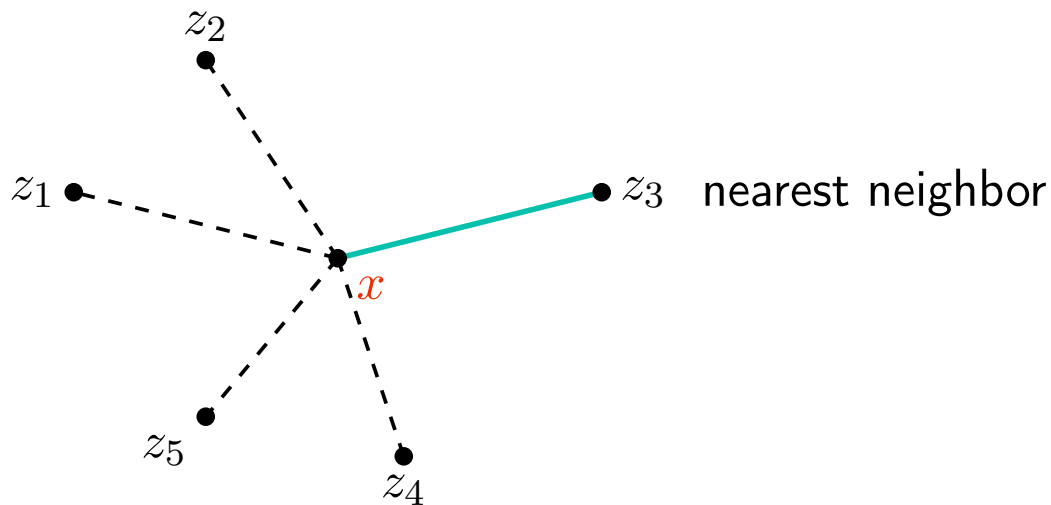
Note: Without this outlier, ℓ_2 and ℓ_1 give nearly identical fits. The outlier is the stress test.

Feature distance and nearest neighbors

- if x, y are feature vectors for two entities, $\|x - y\|$ is the *feature distance*
- for vectors $z_1, \dots, z_m$, z_j is nearest neighbor of x if

$$\|x - z_j\| \leq \|x - z_i\|, \quad i = 1, \dots, m$$

$$p = \begin{cases} 1 \\ 2 \\ \infty \end{cases}$$



$$\|x\|_p$$

$$\|x\|_2 = \sqrt{x_1^2 + \dots + x_n^2}$$

$$\|x\|_1 = |x_1| + \dots + |x_n|$$

$$\|x\|_\infty = \max \{|x_i|\}$$

Example: document dissimilarity

- Suppose we have m articles from Wikipedia: e.g., topics such as 'Veterans Day', 'Memorial Day', 'Academy Awards', 'Golden Globe Awards', 'Super Bowl'
- dictionary of n words: e.g.,
 $\text{dict} = \{ \text{"in"}, \text{"world"}, \text{"red"}, \text{"blue"}, \text{"oscar"}, \text{"football"}, \dots \}$
- word counts of each article as "feature vectors": for each $i \in \{ \text{'Veterans Day'}, \text{'Memorial Day'}, \text{'Academy Awards'}, \text{'Golden Globe Awards'}, \text{'Super Bowl'}, \dots \}$

$$x_i = \begin{bmatrix} \# \text{'in'} \text{ appears} \\ \# \text{"world"} \text{ appears} \\ \vdots \\ 1 \end{bmatrix}$$

- **Question:** How similar are two articles?

Example: document dissimilarity

	$i = 1$ Veterans' day	$i = 2$ Memorial day	$i = 3$ Academy A.	$i = 4$ Golden G.	$i = 5$ Super Bowl
Veterans' day	$\ x_1 - x_1\ $	$\ x_1 - x_2\ $	$\ x_1 - x_3\ $	$\ x_1 - x_4\ $	$\ x_1 - x_5\ $
Memorial day	$\ x_2 - x_1\ $	$\ x_2 - x_2\ $	$\ x_2 - x_3\ $	$\ x_2 - x_4\ $	$\ x_2 - x_5\ $
Academy A.	$\ x_3 - x_1\ $	$\ x_3 - x_2\ $	$\ x_3 - x_3\ $	$\ x_3 - x_4\ $	$\ x_3 - x_5\ $
Golden G.	$\ x_4 - x_1\ $	$\ x_4 - x_2\ $	$\ x_4 - x_3\ $	$\ x_4 - x_4\ $	$\ x_4 - x_5\ $
Super Bowl	$\ x_5 - x_1\ $	$\ x_5 - x_2\ $	$\ x_5 - x_3\ $	$\ x_5 - x_4\ $	$\ x_5 - x_5\ $

Example: document dissimilarity

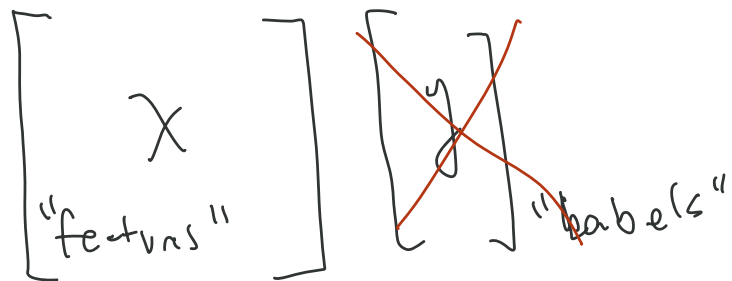
- pairwise distances shown below:

	Veterans' day	Memorial day	Academy A.	Golden G.	Super Bowl
Veterans' day	0	0.095	0.130	0.153	0.170
Memorial day	0.095	0	0.122	0.147	0.181
Academy A.	0.130	0.122	0	0.108	0.164
Golden G.	0.153	0.147	0.108	0	0.164
Super Bowl	0.170	0.181	0.164	0.164	0

Document Dissimilarity

- We may want to cluster documents into groups.
- e..g, awards, sports, holidays, etc.
- We can use k-means to cluster documents into k groups.

What is K-Means Clustering?



- An **unsupervised learning algorithm** for clustering data.
- Goal: partition n observations into k clusters.
- Each observation belongs to the cluster with the nearest mean.

How Does It Work?

Algorithm Conceptual Steps:

1. Choose # of clusters k
 2. Initial set centroids for the k clusters
 3. Assignment step: assign each data pt. $x_i \in \mathbb{R}^d$ to nearest centroid
 4. Update centroid
 - 5 Repeat until convergence
- " $a_1 + 1$ " }*

Key properties:

- objective L decreases
 - guarantee convergence to local min
 - Result depends on initial condition
- k-means ++*

Objective Function: Using Sum of Square Errors (Norms)

$$\mu_j \in \mathbb{R}^d, \quad \underbrace{x_i \in \mathbb{R}^d}_{x_1, \dots, x_n}$$

Within-Cluster Sum of Squares (WCSS)

$$\mathcal{L}(\{\mu_j\}_{j=1}^k, \{z_i\}_{i=1}^n) = \sum_{i=1}^k \sum_{x \in C_i} \|x - \mu_i\|^2 = \sum_{i=1}^n \|x_i - \mu_{z_i}\|^2 \quad \text{VMCS}$$

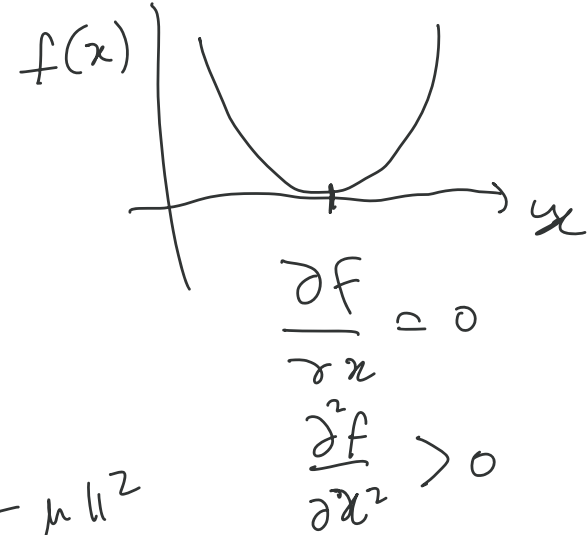
$C_i = \text{cluster } i \in [1, \dots, k]$, μ_i is its centroid

Centroid Update Rule

$$x_1, x_5, x_7 \rightarrow k=i$$

Given cluster assignments at iteration t , each centroid is updated to be the mean of the points in its cluster:

$$\mu_i^{(t+1)} = \frac{1}{|C_i^{(t)}|} \sum_{x \in C_i^{(t)}} x$$

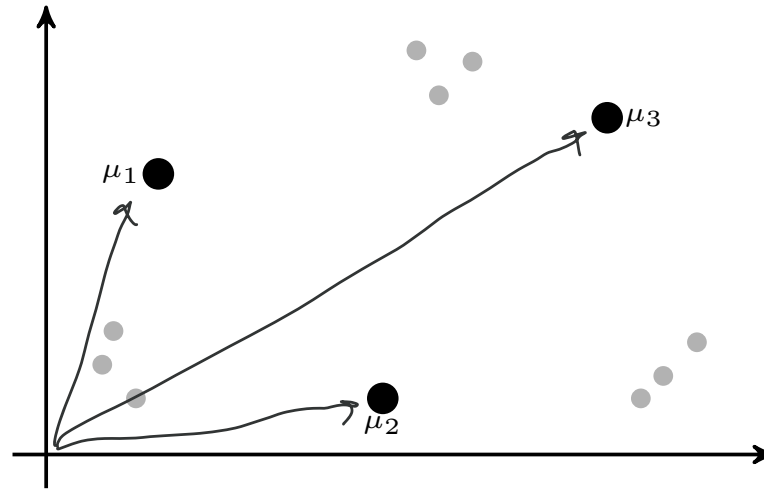


Why the mean?

$$\mu_i^* = \operatorname{argmin}_{\mu} \sum_{x \in C_i^{(t)}} \|x - \mu\|^2$$

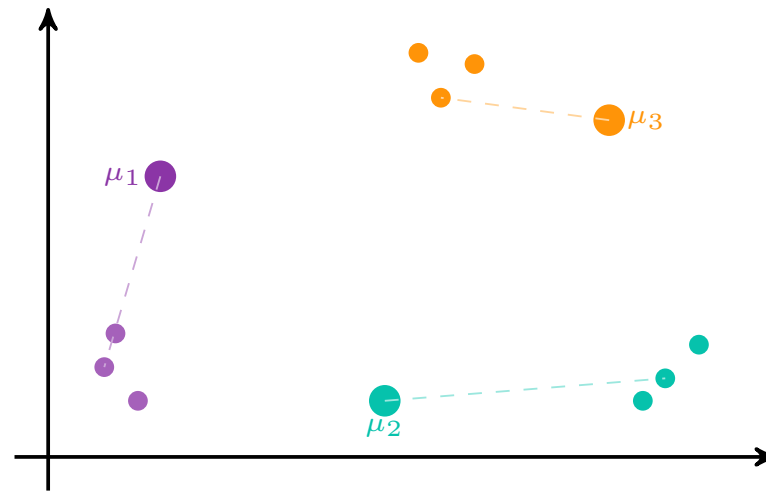
K-Means Algorithm: Initialization

Initialization



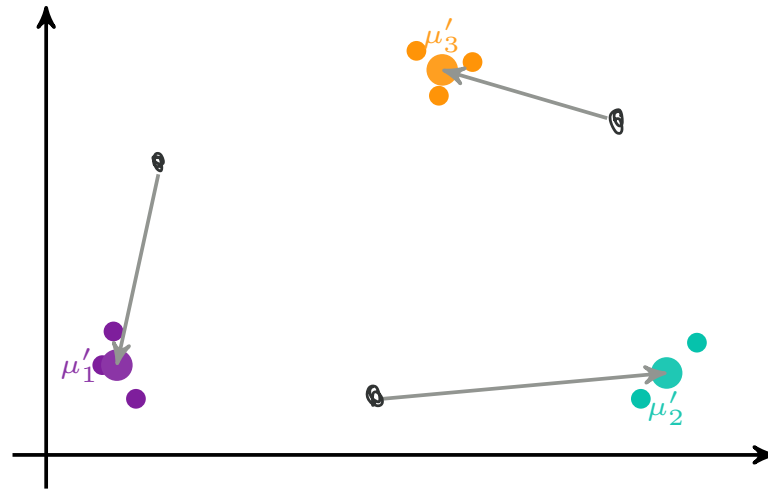
K-Means Algorithm: Iteration Steps

Iteration 1: Assign

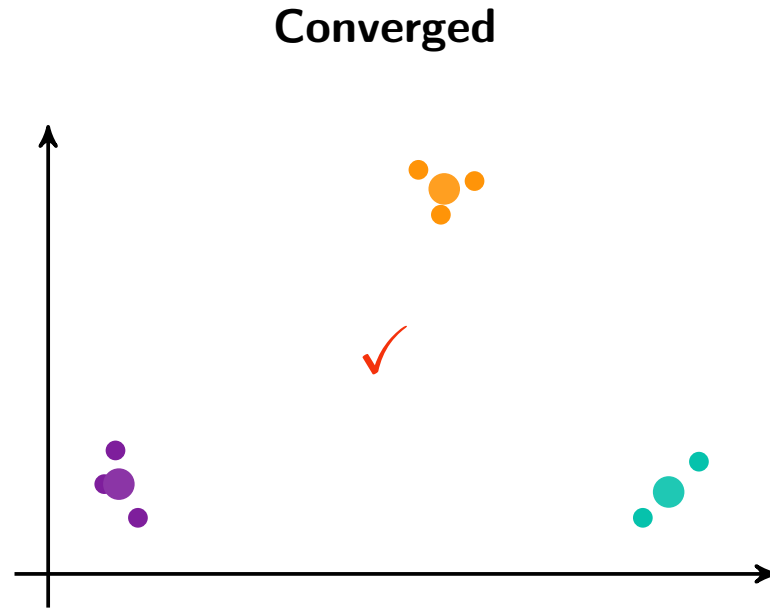


K-Means Algorithm: Iteration Steps

Iteration 1: Update



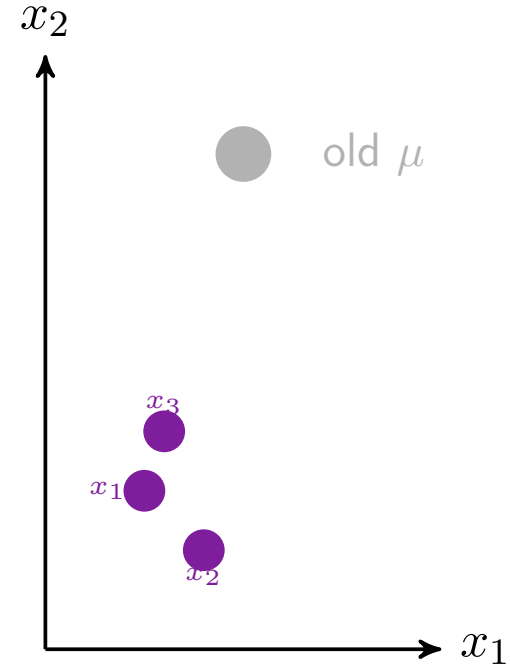
K-Means Algorithm: Iteration Steps



How Centroids Update: Computing the Mean

Update rule:

Example:



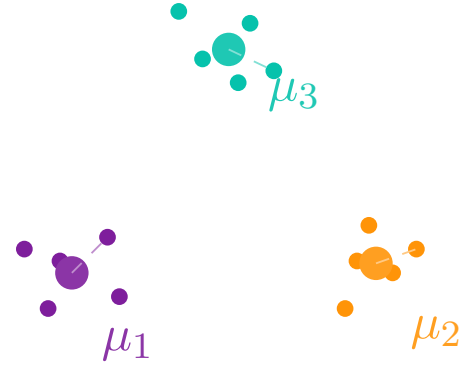
New centroid:

K-Means Algorithm Animation

Summary: k -Means Algorithm

Goal. Partition data into k clusters by minimizing

$$\mathcal{J}(\{\mu_j\}_{j=1}^k, \{z_i\}_{i=1}^n) = \sum_{i=1}^n \|x_i - \mu_{z_i}\|^2$$



Algorithm (Lloyd's algorithm):

1. Initialize μ_j (k -means ++)

2. Repeat:

a. Assignment: $z_i \leftarrow \underset{j \in \{1, \dots, k\}}{\operatorname{argmin}} \|x_i - \mu_j\|^2$

b. Update centroids

$$\mu_j = \frac{1}{|C_j|} \sum_{i \in C_j} x_i$$

K-Means with Different Norms

Real World Impact:

Robust to outliers

- Urban delivery: Use l_1 (Manhattan) because you follow streets
- Air travel: Use l_2 (Euclidean) because you fly straight
- Chess moves: Use l_∞ (king moves anywhere in 8 directions equally)

Norm	Distance metric	Optimal centroid	Cluster shape
l_2	Euclidean $\sqrt{\sum x_i^2}$	Mean	Round / Spherical
l_1	Taxi distance	Componentwise Median	Diamond shape
l_∞	Worst case	Componentwise midrange	Square / Rect-shaped

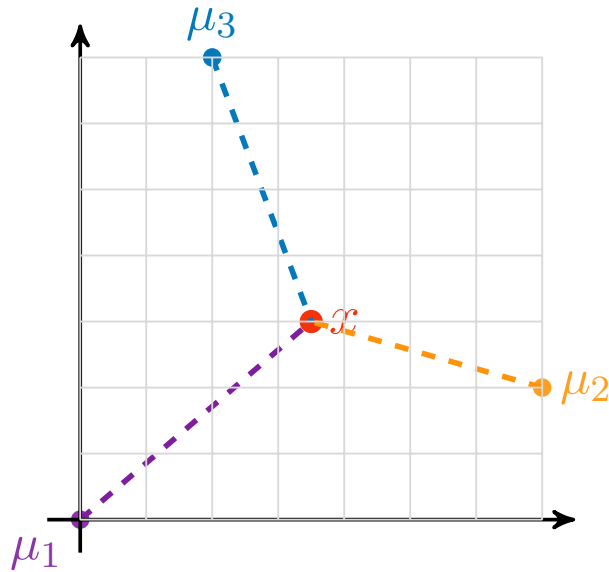
Why l_2 is standard:

$$\mu_j = \frac{1}{|C_j|} \sum_{x \in C_j} x$$

Norms Can Disagree on Assignment

Centroids: $\mu_1 = (0, 0)$ (purple), $\mu_2 = (7, 2)$ (orange), $\mu_3 = (2, 7)$ (cerulean)

Test point: $x = (3.5, 3)$



Norm	$\ x - \mu_1\ $	$\ x - \mu_2\ $	$\ x - \mu_2\ $
ℓ_2			
ℓ_1			
ℓ_∞			

Effect of Distance on K-Means

Euclidean (ℓ_2):

- Clusters are round (balls).
- Center = arithmetic mean.
- Sensitive to scale of features.

Manhattan (ℓ_1):

- Clusters look diamond-shaped.
- Center = componentwise median.
- More robust to outliers.

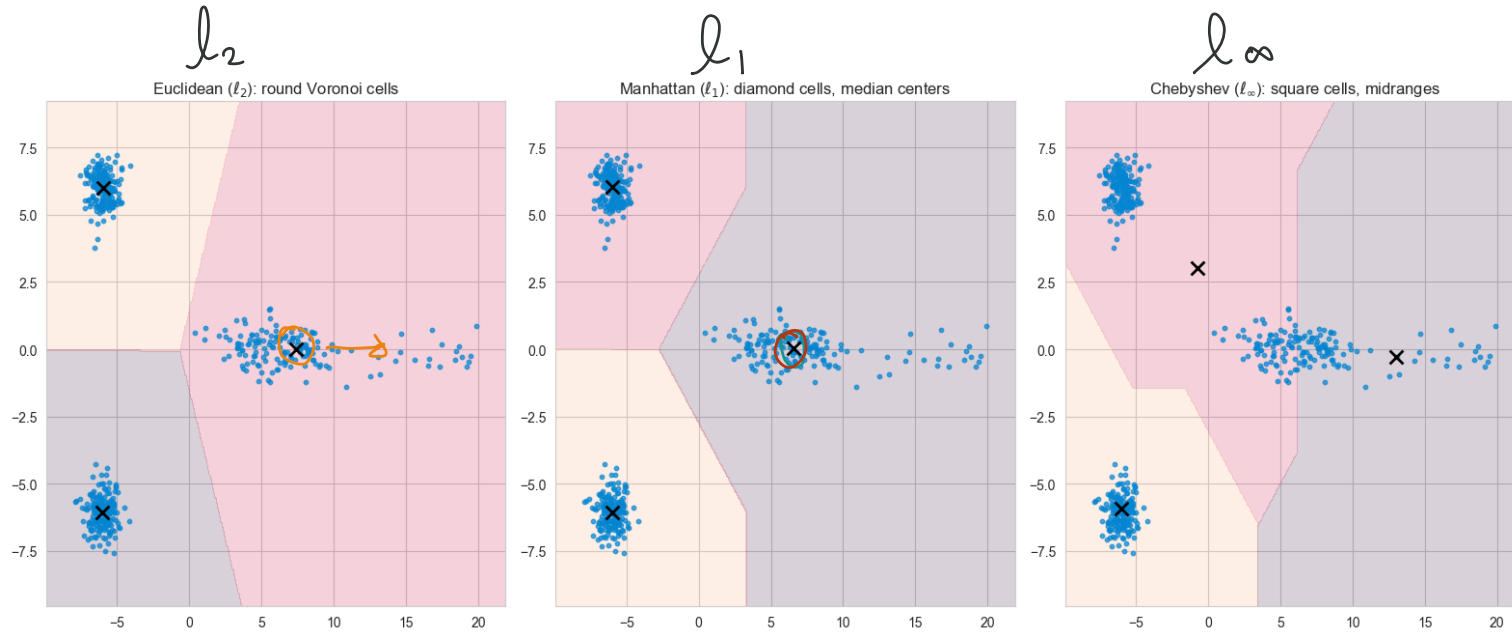
Cosine similarity:

- Clusters by angle, not magnitude.
- Center = normalized mean direction.
- Common for text or embeddings.

General lesson:

- Choice of distance = inductive bias.
- Different geometries $\Rightarrow$ different clusters.

K-Means with Different Distances



$$\min_{\mu} \max_{\delta} \|x + \delta - \mu\|$$

- ℓ_2 : round clusters, center on the elongated cluster is pulled toward the outliers (mean).
- ℓ_1 : diamond clusters, center sits closer to the componentwise median, resisting the long tail.
- ℓ_∞ : angular clusters, center sits at the midrange (avg of min/max per coordinate), so it shifts differently again.

How to Choose the Number of Clusters k ?

The challenge: k is a hyperparameter that must be specified before running k-means.

Common approaches:

1. **Domain knowledge:** Sometimes k is known from the problem
 - ▶ Customer segments, product categories, etc.
2. **Elbow method:** Plot objective $\mathcal{L}(k)$ vs. k , look for “elbow”
 - ▶ Most popular heuristic
 - ▶ Where adding more clusters gives diminishing returns
3. **Silhouette score:** Measure cluster quality
 - ▶ How well points fit their clusters vs. other clusters
4. **Gap statistic:** Compare to random data
 - ▶ More principled but computationally expensive

Reality: Often no single “correct” k ; depends on application.

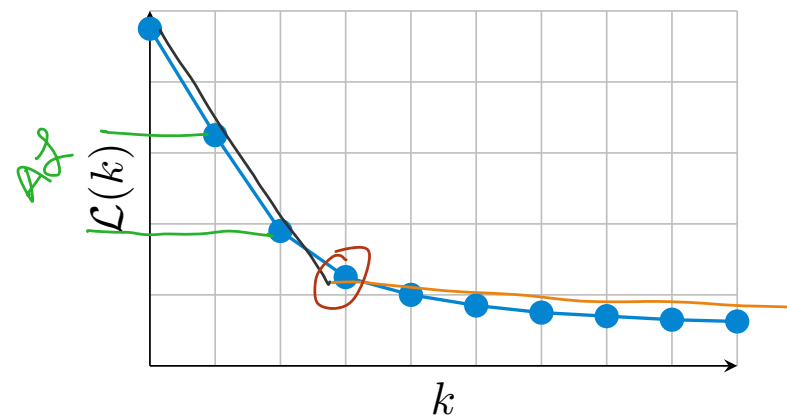
The Elbow Method

$$\{\mu_j\}_{j=1}^k \dots$$
$$\mathcal{L}(k) = \sum_{i=1}^n \|x_i - \mu_{z_i}\|^2$$

Idea: Plot $\mathcal{L}(k)$ vs. k and look for the “elbow.”

Why does $\mathcal{L}(k)$ decrease?

- point where curve bends
- Diminishing returns



Silhouette Score

Idea: Measure how well each point fits its cluster.

For each point i :

Silhouette coefficient:

Interpretation:

Average silhouette score:

Choosing k : Summary

Method	Pros	Cons
Elbow	Simple, intuitive, fast	Elbow not always clear; subjective
Silhouette	Quantitative measure of cluster quality	More computation; may favor compact clusters
Gap statistic	Principled comparison to null model	Expensive; requires many bootstrap samples
Domain knowledge	Most reliable when available	Not always available

Best practice:

- Try multiple methods
- Visualize results for different k
- Consider the application: what granularity makes sense?
- Remember: clustering is exploratory!

k-NN vs k-Means

$\begin{bmatrix} VD \\ MD \\ AA \\ GL \\ SB \end{bmatrix}$ $\begin{bmatrix} H \\ H \\ AS \\ AS \\ S \end{bmatrix}$

k-Nearest Neighbors (k-NN)

- Supervised learning algorithm
- Used for classification (and regression)
- Predicts label of a new point by majority vote of its k nearest neighbors
- Relies on a distance metric (Euclidean, Manhattan, etc.)
- No training phase beyond storing the data

$\begin{bmatrix} X \end{bmatrix} \begin{bmatrix} Y \end{bmatrix}$
"label" $\in \{S\}$

k-Means Clustering

- Unsupervised learning algorithm
- Used for clustering unlabeled data
- Iteratively assigns points to k clusters and updates cluster centers
- Objective: minimize within-cluster variance
- Training = alternating assignment and update until convergence

Both use k and a distance metric, but serve very different purposes.

k-Nearest Neighbors (k-NN)

Idea: Classify a new point based on the labels of its k nearest neighbors.

predict a value for new point based on values of k-NN

Key properties:

- **Supervised learning** algorithm (requires labeled training data)
- Used for both classification and regression
- **Non-parametric:** No explicit model/parameters to learn
- **Instance-based:** Stores entire training dataset
- **Lazy learning:** No training phase; all computation at prediction time

$y = \theta_0 + \theta_1 x_1 + \theta_2 x_2$

(The equation is crossed out with a large X, indicating it is not applicable to k-NN.)

Decision rule (classification):

$$\hat{y} = \text{majority vote among } k \text{ nearest neighbors of } x$$

Example: Given a new email, is it spam or not spam? Look at the k most similar emails in your database and take a vote.

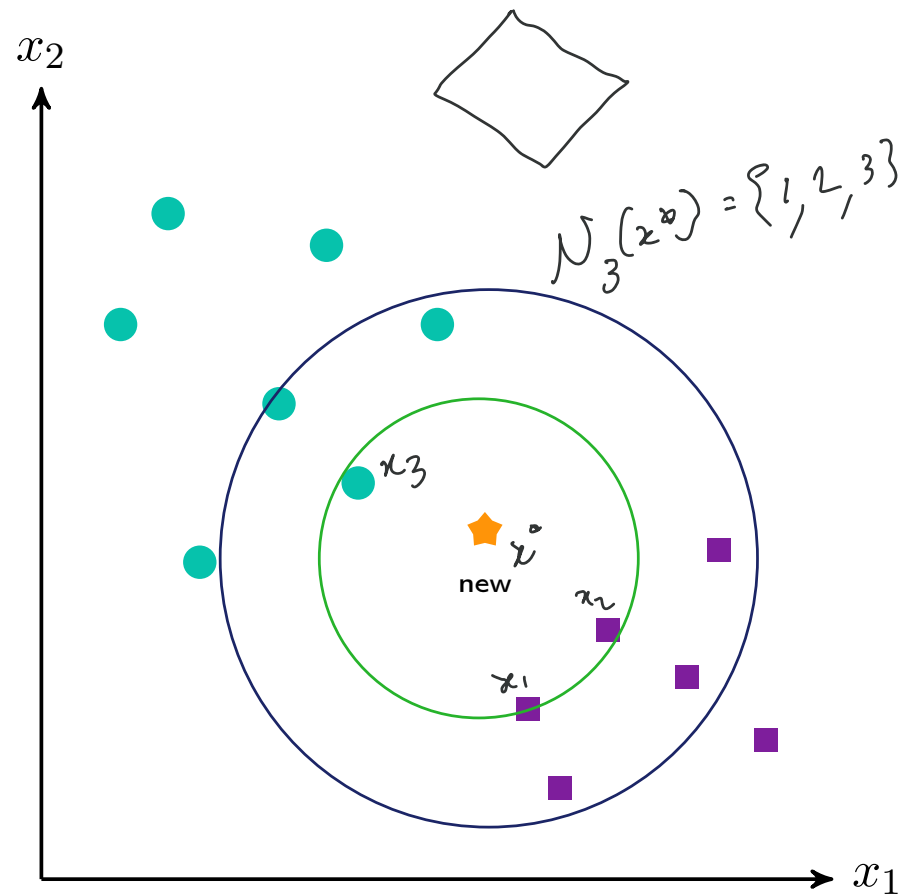
k-NN Classification: Visual Example

Setup:

- Training data: red circles, blue squares
- New point: green star
- Question: What class?

$k=3$
2 purple dots
1 turquoise dot
} $\Rightarrow$ new pt. is purple.

$k=4$
3 turquoise
5 purple
} $\Rightarrow$ new pt. purple



k-NN Algorithm

Training phase:

1. Store all your data points (x_i, y_i)

$$N_k(x^*) = \{i \in [1, \dots, n] :$$

x_i is a k-NN
of x^*

Prediction phase: Given a new point x^* ,

1. Compute the distance from x^* to all points

$$m_i = \|x^* - x_i\| \quad i = 1, \dots, n$$

2. Find the k pts. w/ smallest distance

$$N_k(x^*) = \{i : i \text{ is index of k-NN}\}$$

$$m_i = \|x^* - x_i\| \leq m_j = \|x^* - x_j\|$$

majority vote

3. Output: [classification]

$$\hat{y} = \arg \max_c \sum_{i \in N_k(x^*)} \mathbb{1}_{\{y_i = c\}}$$

[Regression]: $\hat{y} = \frac{1}{k} \sum_{i \in N_k(x^*)} y_i$

$$\frac{1}{k} \sum_{i \in N_k(x^*)} y_i$$

k-NN: Classification vs Regression

Same algorithm, different outputs depending on the task

Classification (discrete labels)

Goal: Predict category/class

Output: Label from finite set

- Spam/Ham
- Dog/Cat/Bird
- Digit 0-9

Decision rule:

$$\hat{y} = \text{mode}\{ y_i : i \in \mathcal{N}_k(x) \}$$

(most frequent, majority vote among k neighbors)

Regression (continuous values)

Goal: Predict numeric value

Output: Real number

- House price
- Temperature
- Stock price

Decision rule:

$$\hat{y} = \frac{1}{k} \sum_{i \in \mathcal{N}_k(x)} y_i$$

(average of k neighbors)

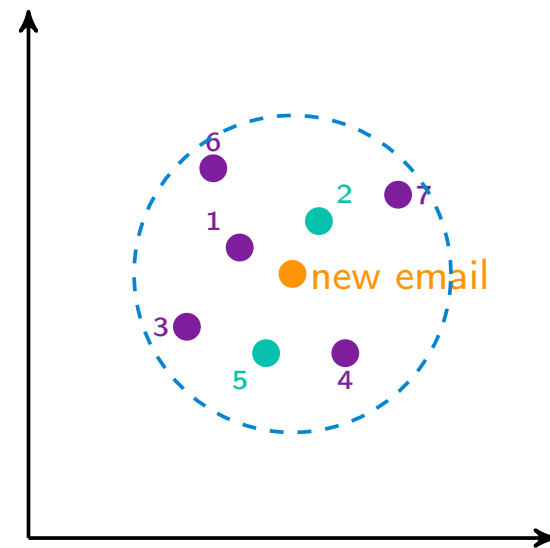
Let's Look at Examples

k-NN Classification: Understanding Majority Vote

Example: Email classification (Spam vs Ham) with $k = 7$

Step 1: Find 7 nearest neighbors

Neighbor	Distance	Label
1	0.12	Ham
2	0.18	Spam
3	0.21	Ham
4	0.25	Ham
5	0.29	Spam
6	0.31	Ham
7	0.35	Ham



Step 2: Count votes

5 HAMs, 2 SPAMs $\Rightarrow$ New email is SAFE

generate prediction:

Vote tally:

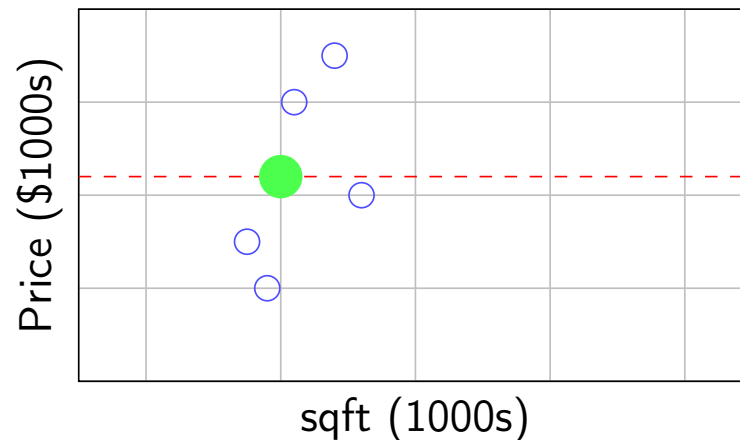
k-NN for Regression: Predicting Continuous Values

Example: Predict house price based on similar houses

New house: 1800 sqft, 3 bedrooms

Find $k = 5$ nearest neighbors:

House	Distance	Price
A	0.08	\$250k
B	0.12	\$280k
C	0.15	\$240k
D	0.18	\$290k
E	0.22	\$260k



Prediction (average):

$$\hat{y} = \frac{(250 + \dots + 260)}{5} = 264$$

Weighted k-NN: Distance Matters

Distance: SPAM/HAM

$$C = \text{HAM} \quad \sum_{i \in N_k(x^*)} w_i \mathbb{1}[y_i(x_i) = \text{HAM}]$$

$$C = \text{SPAM} \quad = \frac{1}{0.1} \cdot \mathbb{1}[y_1(x_1) = \text{HAM}]$$

Problem: All k neighbors vote equally, even if some are much farther away.

Solution: Weight votes by distance

Unweighted (standard): All neighbors vote equally

$$\hat{y} = \underset{c}{\operatorname{argmax}} \sum_{i \in N_k(x^*)} \mathbb{1}\{y_i = c\}$$

Example: $k = 3$

$x_1: d = 0.1, \text{HAM}$
 $x_2: d = 0.5, \text{SPAM}$
 $x_3: d = 0.9, \text{SPAM}$
 $\Rightarrow \text{SPAM}$

Weighted: $w_i = \frac{1}{d_i}$ or $w_i = \frac{1}{d_i^2}$

$$\hat{y} = \underset{c}{\operatorname{argmax}} \sum_{i \in N_k(x^*)} w_i \mathbb{1}[y_i = c]$$

$\frac{1}{0.1} \mathbb{1}[y_1(x_1) = \text{HAM}] + \frac{1}{0.5} \mathbb{1}[y_2(x_2) = \text{SPAM}] + \frac{1}{0.9} \mathbb{1}[y_3(x_3) = \text{SPAM}]$
 $= 10 + 2 + 1 = 13$
 $\Rightarrow \text{HAM}$

Same example: weights $w_i = 1/d_i$

$$w_i = 1/d_i$$

$\text{HAM} : 1/0.1 = 10$
 $\text{SPAM} : \frac{1}{0.5} + \frac{1}{0.9} = 2.1$
 $\Rightarrow \text{HAM}$

Weighted K-NN: Distance as a Weighted Norm $\| \cdot \|_2$

STANDARD k-NN : $\text{dist}(x, x_i) = \|x - x_i\|_2 = \sqrt{\sum_{j=1}^d (x_j - x_{i,j})^2}$

Weighted k-NN

$$\text{dist}_w(x, x_i) = \|x - x_i\|_w = \sqrt{\sum_{j=1}^d w_j (x_j - x_{i,j})^2}$$

$w_j > 0$ are feature weights

$$W = \text{diag}(w_1, w_2, \dots, w_d)$$

$$= \begin{bmatrix} w_1 & 0 & \dots & 0 \\ 0 & w_2 & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & \dots & 0 & w_d \end{bmatrix}$$

$$\text{dist}_w(x, x_i) = \sqrt{(x - x_i)^T W (x - x_i)}$$

$$\|v\|_w = \sqrt{v^T W v}$$

Handling Ties in k-NN

Problem: What if neighbors vote equally for multiple classes?

Strategies for breaking ties:

1. **Use odd k** (for binary classification)
 - ▶ Guarantees no 50-50 split
 - ▶ But can still tie with 3+ classes
2. **Pick the class of the nearest neighbor**
 - ▶ Among tied classes, choose the one with smallest min distance
3. **Random choice**
 - ▶ Randomly select among tied classes
4. **Use weighted voting**
 - ▶ Weight by $1/d_i \rightarrow$ ties are rare
5. **Reduce k**
 - ▶ Use $k - 1$ neighbors instead

Best practice: Use odd k and/or weighted voting

Choosing k in k-NN

$$\left\{ \begin{bmatrix} x \\ y \end{bmatrix} \right\}$$

Important: k is the number of neighbors to consider for prediction.

Small k (e.g., $k = 1$):

- Very flexible decision boundary $\implies$ Can overfit
- Low bias, high variance $\implies$ Sensitive to noise

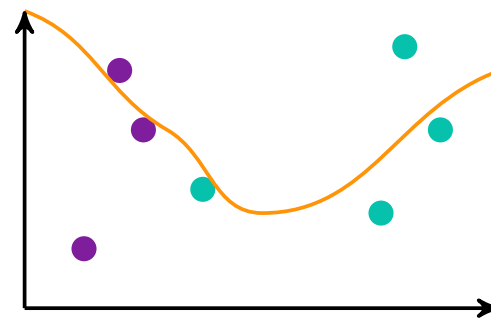
Large k :

- Smoother decision boundary $\implies$ Can underfit
- High bias, low variance $\implies$ More robust to noise

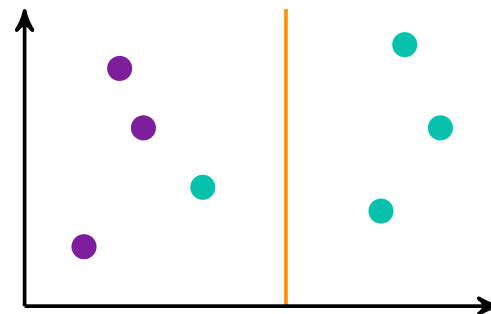
Typical choice: $k = \sqrt{n}$ (rule of thumb)

- Use cross-validation
- Odd k for binary classification (avoid ties)

$k = 1$ (**Overfit**)



$k = 7$ (**Better**)



Distance Metrics in k -NN

Metric	Formula	When to use
l_2 (Euclidean)	$\ x\ _2 = \sqrt{x_1^2 + \dots + x_d^2}$ $\ x-y\ _2$	• Continuous features w/ similar scaling • Common choice • General purpose
l_1 (taxi cab)	$\ x\ _1 = x_1 + \dots + x_d $ $\ x-y\ _1$	• Grid like (city blocks) • High dimensional • Robust outliers
Cosine $d_c(x,y)$	$d_c(x,y) = 1 - \frac{x^T y}{\ x\ \ y\ }$	• Text data (word counts) • High dim. or sparse
l_∞ (word case) Chebyshev	$\max_i x_i - y_i $	• When all features matter equally

Why Feature Scaling Matters for k-NN

Example: Predicting house price using area and # bedrooms

Without scaling:

Training data:

- House A: (2000 sqft, 3 beds)
- House B: (1500 sqft, 2 beds)

New house: (1900 sqft, 2 beds)

Distance to A:

$$m = \sqrt{(1900 - 2000)^2 + (2 - 3)^2}$$
$$= \sqrt{100^2 + 1^2} \approx 100$$

Area dominates! Bedrooms barely matter.

With scaling:

Standardized:

- House A: (1.2, 0.8)
- House B: (-0.6, -0.4)

New house: (0.9, -0.4)

Distance to A:

$$m = \sqrt{0.09 + 1.44} \approx 1.24$$

Both features contribute equally.

Always standardize features for k-NN!

k-NN vs k-Means: Key Differences

	k-NN	k-Means
Type	Supervised (needs labels)	Unsupervised (no labels)
Task	Classification/Regression	Clustering
What is k?	Number of neighbors to consider	Number of clusters
Training	None (stores data)	Iterative optimization
Prediction	Expensive (compute all distances)	Fast (assign to nearest centroid)
Output	Label or value for new point	Cluster assignments for all points
Memory	High (stores all training data)	Low (only stores k centroids)

Both:

- Use distance metrics (Euclidean, Manhattan, etc.)
- Sensitive to feature scaling
- Require choosing k (but means different things!)

Example: Document Classification with k-NN

Task: Classify a new Wikipedia article by topic

Setup:

- Feature vector: word count histogram ($n = 4423$ words)
- Distance: Euclidean distance on normalized counts
- $k = 3$ nearest neighbors

Training data:

- Veterans Day → Holiday
- Memorial Day → Holiday
- Academy Awards → Award Show
- Golden Globes → Award Show
- Super Bowl → Sports

New article: “Labor Day”

1. Compute distances to all 5 training articles
2. 3 nearest: Veterans Day (0.08), Memorial Day (0.10), Academy Awards (0.15)
3. Majority vote: 2 Holiday, 1 Award
4. **Prediction: Holiday ✓**

Document Classification: k-NN vs k-Means

Example corpus:

Veterans Day, Memorial Day, Academy Awards, Golden Globes, Super Bowl

k-Means (unsupervised)

- Input: pairwise document distances (from word-count histograms).
- Goal: cluster into $k = 3$ groups.
- Outcome:
 - ▶ Veterans Day, Memorial Day → Holidays
 - ▶ Academy Awards, Golden Globes → Awards
 - ▶ Super Bowl → Sports
- No labels required.

k-NN (supervised)

- Assume we already have labels: Holiday, Award Show, Sports.
- New document: “Labor Day.”
- Compute distances to all 5 documents.
- Nearest neighbors (Veterans Day, Memorial Day).
- Majority label = Holiday $\Rightarrow$ classify as Holiday.

Same distance representation, two different uses.

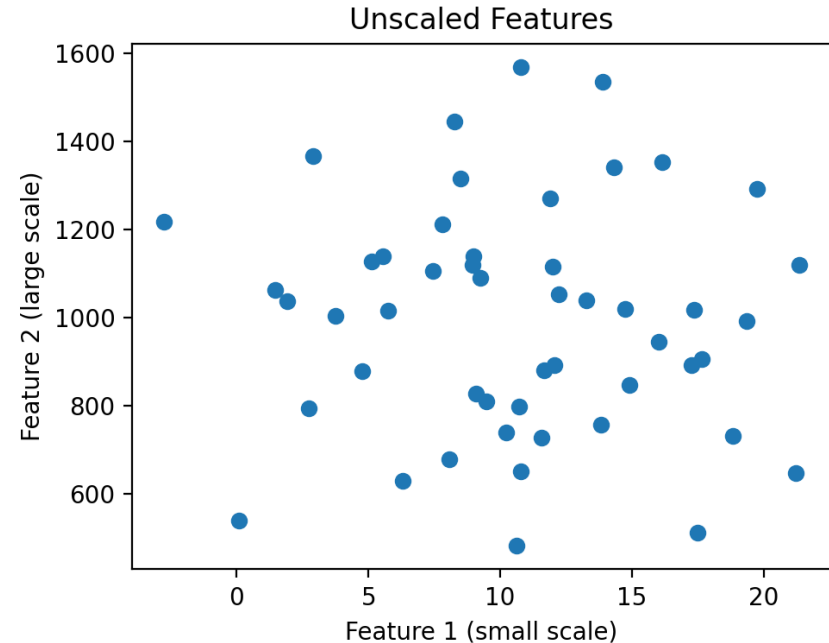
(If time permits) Standardizing Data

Motivation: Why Standardize?

- Raw features often have different units and scales. e.g. height (cm) vs. income (\$).
- Algorithms that rely on distances or gradients can be biased by scale.
- Unscaled features $\rightarrow$ distort optimization and distance metrics.

Consequences:

- Gradient descent converges slowly or doesn't at all.
- Features with large magnitudes dominate loss.
- Poorly shaped decision boundaries.



Definition and Formula

Standardization

Transforms features to have zero mean and unit variance:

$$z = \frac{x - \mu}{\sigma}$$

- Centers data around 0 and rescales by standard deviation.
- Applied feature-wise across samples.

Comparison

Method	Formula	Range
Standardization	$(x - \mu)/\sigma$	Centered (mean 0)
Normalization	$(x - \min)/(\max - \min)$	[0, 1]

Example: Mean as Inner Product

- The mean is the "balance point" (center of mass).
- The sample mean is

$$\mu = \bar{x} = \frac{1}{n} \sum_{i=1}^n x_i = \frac{\langle x, \mathbf{1} \rangle}{\langle \mathbf{1}, \mathbf{1} \rangle} = \frac{x^T \mathbf{1}}{\mathbf{1}^T \mathbf{1}}$$

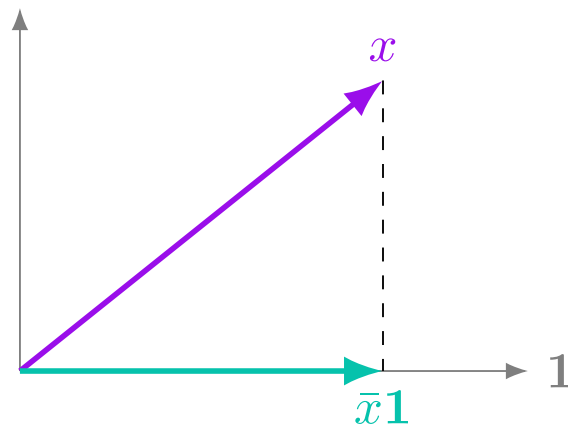
$$\mathbf{1} = (1, 1, \dots, 1)$$

$$x = [x_1, x_2, \dots, x_d]$$

$$\begin{bmatrix} 1 \\ 1 \\ \vdots \\ 1 \end{bmatrix}$$

- Geometrically: $\bar{x} \cdot \mathbf{1}$ is the **projection** of x onto the "span" of $\mathbf{1}$
- Intuition: the mean is the "best constant approximation" of the data

$$(\bar{x}, \bar{x}, \dots, \bar{x})$$



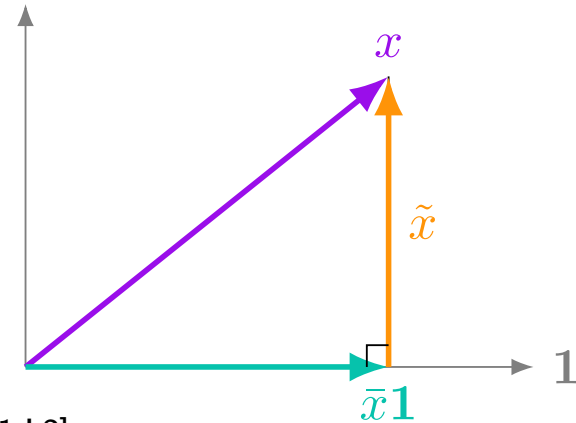
Centering Feature Vectors

Given $x \in \mathbb{R}^n$ and $\mathbf{1} = (1, 1, \dots, 1)^\top \in \mathbb{R}^n$:

$$\begin{bmatrix} \alpha_1 & \alpha_2 \\ \alpha_3 & \alpha_4 \end{bmatrix}^\top = \begin{bmatrix} \alpha_1 & \alpha_3 \\ \alpha_2 & \alpha_4 \end{bmatrix}$$

$$\tilde{x} = x - \underbrace{\bar{x} \mathbf{1}}_{\underbrace{\frac{1}{n} \mathbf{1}^\top x \mathbf{1}}_{\frac{1}{n} \mathbf{1}^\top x}} = x - \underbrace{\frac{1}{n} \mathbf{1} \mathbf{1}^\top}_{\left(I - \frac{1}{n} \mathbf{1} \mathbf{1}^\top\right)} x$$

- $\mathbf{1}\mathbf{1}^\top \in \mathbb{R}^{n \times n}$ is the all-ones matrix.
- $H = I - \frac{1}{n} \mathbf{1}\mathbf{1}^\top$ is the **centering matrix**.
- H is symmetric and idempotent ($H^2 = H$): an **orthogonal projection**.
- Geometrically: projects x onto the subspace orthogonal to $\mathbf{1}$.



Example: Variance and Standard Deviation

- The centered data vector is $x - \mu \mathbf{1}$
- The standard deviation is the "typical distance" of points from that balance point.
 - ▶ Small SD $\rightarrow$ data clustered tightly around the mean.
 - ▶ Large SD $\rightarrow$ data spread out.
- If you pick a random data point, the standard deviation is roughly the size of the "typical error" you'd make if you always guessed the mean.
- Think of the mean as a reference position. The standard deviation is like the moment of inertia—how far mass is spread out from the center. Same "center," but very different spread.
- The variance is the **scaled average squared deviation**

Example: Variance and Standard Deviation

- The centered data vector is $\tilde{x} = x - \bar{x}\mathbf{1} = Hx$, where $H = I - \frac{1}{n}\mathbf{1}\mathbf{1}^\top$ is the centering matrix.
- Its squared norm gives the total squared deviation:
- The variance is the **scaled average squared deviation**:
- Standard deviation (= avg length of the projected (centered) vector):

Standardizing Data

Given data $x = (x_1, \dots, x_n)$ with mean $\bar{x}$ and std. deviation s , the **standardized data** (z-scores) are

Properties

- Centered: $\text{mean}(z) = 0$
- Scaled: $\text{sd}(z) = 1$
- Interpretable in **standard deviation units**.
- Comparable across different variables

Standardizing Data (z-scores)

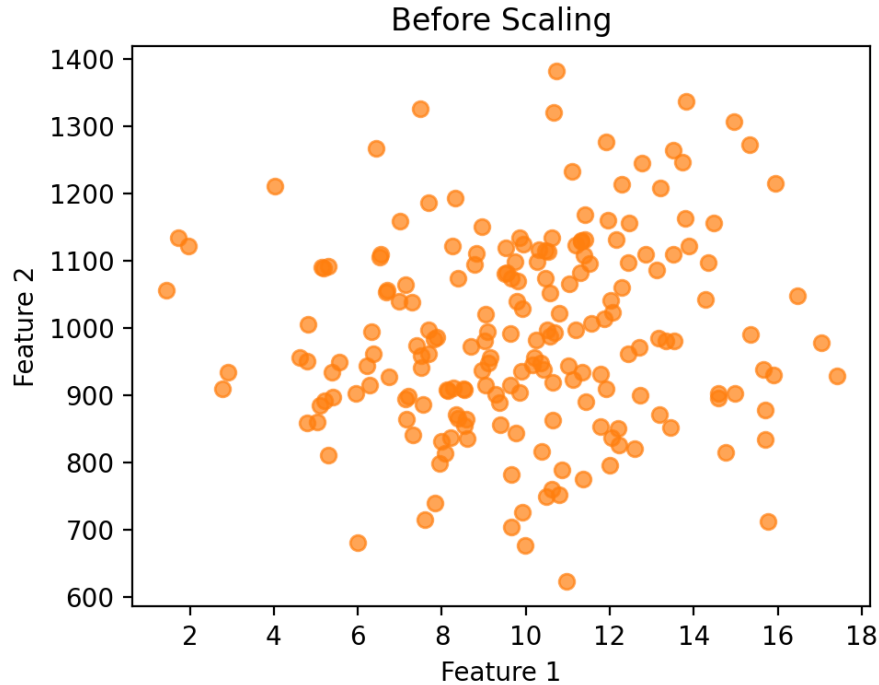
Given $x \in \mathbb{R}^n$, mean $\bar{x}$, and centering matrix the **centered data** is

The **standard deviation** is

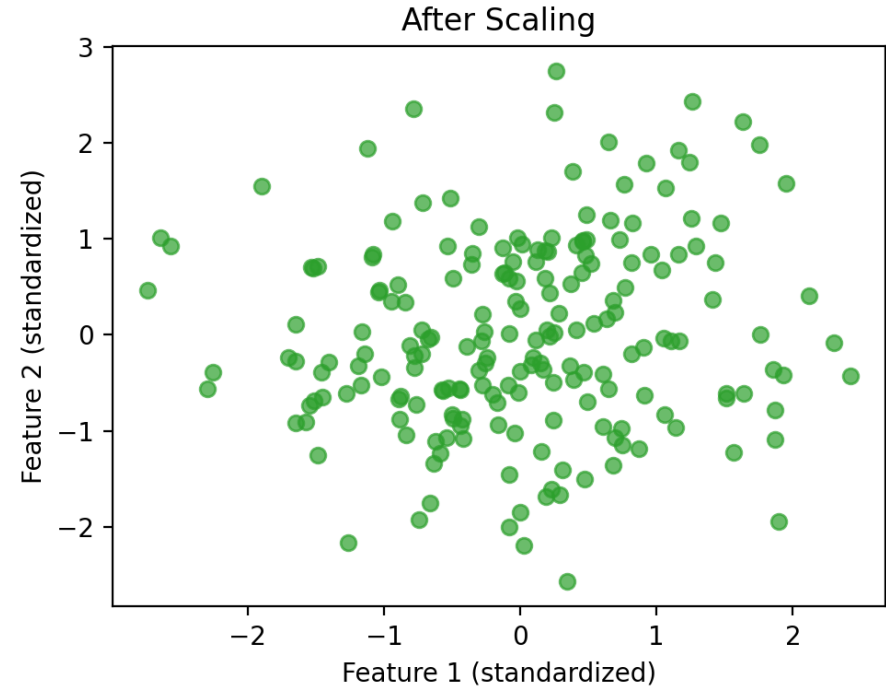
Standardized data (z-scores):

Before and After Scaling (Example Visualization)

Before:



After:



- Scaled features have comparable ranges.
- Improves numerical stability and convergence.

Python Code

```
import numpy as np from
sklearn.preprocessing import
StandardScaler
# Example dataset: 5 samples, 3
features
X = np.array([
[1.0, 200.0, 3.5],
[2.0, 180.0, 2.5],
[3.0, 220.0, 4.0],
[4.0, 210.0, 3.0],
[5.0, 190.0, 3.8] ])
```

```
# Manual computation using NumPy
means = np.mean(X, axis=0)
variances = np.var(X, axis=0,
ddof=0) # population variance
stds = np.sqrt(variances)
# scikit-learn's StandardScaler
scaler = StandardScaler().fit(X)
print("mean:", scaler.mean_)
print("Scaler var_ :", scaler.var_)
# Apply standardization
X_scaled = scaler.transform(X)
```

Which Algorithms Need It?

Scale-sensitive algorithms:

- Linear and Logistic Regression (faster convergence)
- Support Vector Machines (SVM)
- k-Nearest Neighbors (distance-based)
- Principal Component Analysis (PCA)
- Neural Networks (balanced gradients)

Scale-invariant algorithms:

- Decision Trees
- Random Forests
- Gradient Boosted Trees

Why (and When Not) to Standardize Data

- **Why standardize:**
 - ▶ **Comparability across features** Different units (kg, \$, counts) → put in common **std dev units**.
 - ▶ **Geometry matters** kNN, SVM, PCA use distances/dot products; large-scale features dominate otherwise.
 - ▶ **Optimization stability** Gradient descent converges faster; loss surface becomes "rounder."
 - ▶ **Interpretability** Coefficients are comparable across features.

Why (and When Not) to Standardize Data

- **Why standardize:**
 - ▶ **Comparability across features** Different units (kg, \$, counts) → put in common **std dev units**.
 - ▶ **Geometry matters** kNN, SVM, PCA use distances/dot products; large-scale features dominate otherwise.
 - ▶ **Optimization stability** Gradient descent converges faster; loss surface becomes "rounder."
 - ▶ **Interpretability** Coefficients are comparable across features.
- **When **not** to standardize:**
 - ▶ **Categorical features** encoded as one-hot or integers → standardizing breaks their meaning.
 - ▶ **Features with physical meaning in the model** e.g. age in years, probability values, counts with direct interpretability.
 - ▶ **Tree-based models** (decision trees, random forests, gradient boosting) → splits are order-based, not distance-based; scaling irrelevant.

Practical Tips and Pitfalls

- Always fit the scaler on the **training set only**.
- Apply the same transformation to validation/test sets:

$$z_{\text{test}} = \frac{x_{\text{test}} - \mu_{\text{train}}}{\sigma_{\text{train}}}$$

- Avoid **data leakage** by not recomputing mean/std on test data.
- In scikit-learn:

```
from sklearn.preprocessing import StandardScaler  
scaler = StandardScaler()  
X_train_std = scaler.fit_transform(X_train)  
X_test_std = scaler.transform(X_test)
```

NEXT TIME

MODULE 2 : Regression (Linear w/ some dabbling in other models)

i.e. predicting continuous valued quantities

time

price

distance